class Clases — Servicios transversales (completo)

«static»
Servicios.Exportacion.SerializadorCsv

- Separador: char {readOnly}
- PrefijosFormula: char[]

+ Generar(encabezados: string[], datos: DataTable): string
- UnirFila(campos: string[]): string
- Escapar(campo: string): string

«static»
Servicios.Multidioma.GestorIdioma

- _observers: IList<IIdiomaObserver>
- _lockObs: object
- _idiomaActual: Idioma
- _tradActuales: Dictionary<string,string>
- _idiomasDisponibles: IList<Idioma>

«Property»
+ IdiomaActual: Idioma
+ TradActuales: Dictionary<string,string>
+ IdiomasDisponibles: IList<Idioma>

+ SetIdiomasDisponibles(idiomas: IList<Idioma>): void
+ SuscribirObservador(observer: IIdiomaObserver): void
+ DesuscribirObservador(observer: IIdiomaObserver): void
+ CambiarIdioma(idioma: Idioma, traducciones: Dictionary<string,string>): void
+ CambiarIdioma(idioma: Idioma): void
- Notificar(idioma: Idioma): void

«interface»
Servicios.Multidioma.IIdiomaObserver

+ UpdateLanguage(idioma: Idioma): void

Servicios.Multidioma.Idioma

«Property»
+ Id: string
+ Nombre: string
+ EsDefault: bool

Servicios.Multidioma.Traduccion

«Property»
+ Clave: string
+ Texto: string

«static»
Servicios.Multidioma.TraductorPerfil

+ Nombre(perfil: string): string

«static»
Servicios.Multidioma.Traductor

- _lockCorpus: object
- _corpus: Dictionary<string,IDictionary<string,Traduccion>>

+ ObtenerIdiomas(): IList<Idioma>
+ ObtenerIdiomasHardcode(): IList<Idioma>
+ ObtenerIdiomaDefault(): Idioma
+ ObtenerTraducciones(idioma: Idioma): IDictionary<string,Traduccion>
+ Resolver(clave: string, fallback: string, args: object[], traducciones: IDictionary<string,Traduccion>): string
+ Resolver(clave: string, fallback: string, args: object[], idioma: Idioma): string
+ ObtenerTraduccionesHardcode(idioma: Idioma): IDictionary<string,Traduccion>
+ InferirFormulario(clave: string): string
- Corpus(): Dictionary<string,IDictionary<string,Traduccion>>
- CargarCorpus(): Dictionary<string,IDictionary<string,Traduccion>>
- Desescapar(s: string): string
- Construir(raw: Dictionary<string,string>): IDictionary<string,Traduccion>

«static»
Servicios.Serializacion.FabricaSerializadores

+ ExtensionXml: string {readOnly}
+ ExtensionJson: string {readOnly}
+ FormatoXml: string {readOnly}
+ FormatoJson: string {readOnly}

+ Crear<T>(rutaOExtension: string): SerializadorAbstracto<T>
+ Formato(ruta: string): string
- EsXml(ruta: string): bool
- EsJson(ruta: string): bool
- ExtensionNoAdmitida(ruta: string): BE.AppException
- Extension(ruta: string): string
- NombreArchivo(ruta: string): string

Servicios.Serializacion.GestorArchivos

+ NombreAplicacion: string {readOnly}
+ SubcarpetaVersiones: string {readOnly}
+ SubcarpetaErrores: string {readOnly}
+ SubcarpetaErroresExportados: string
- _instancia: GestorArchivos
- _carpetaRaiz: string

«Property»
+ Instancia: GestorArchivos
+ CarpetaRaizPorDefecto: string
+ CarpetaRaiz: string

+ Carpeta(subcarpeta: string): string
+ PrepararCarpeta(subcarpeta: string): string
+ NombreArchivoSugerido(prefijo: string, detalle: string, fecha: DateTime, extension: string): string

«Create»
- GestorArchivos()

«static»
Servicios.Serializacion.RegistroErrores

+ MaximoPorArchivo: int {readOnly}
- Candado: object
- NombreMensual: Regex

«Property»
+ Carpeta: string

+ PrepararCarpeta(): string
+ PrepararCarpetaExportados(): string
+ RutaDelMes(fecha: DateTime): string
+ EsArchivoMensual(ruta: string): bool
+ Crear(ex: Exception, modulo: string, usuario: string, fecha: DateTime): BE.RegistroError
+ Registrar(ex: Exception, modulo: string, usuario: string): bool
+ ObtenerTodos(): List<BE.RegistroError>
+ LeerArchivo(ruta: string): BE.LibroErrores
+ Exportar(errores: List<BE.RegistroError>, ruta: string): string
+ SembrarEjemplos(): void
- Ejemplo(fecha: DateTime, usuario: string, modulo: string, tipo: string, mensaje: string, causa: string, equipo: string): BE.RegistroError
- LeerSinFallar(ruta: string): BE.LibroErrores

«abstract»
Servicios.Serializacion.SerializadorAbstracto

Utf8SinBom: Encoding

«Property»
+ Extension: string
+ Formato: string

+ ExigirVersionFormato(leida: int, admitida: int): void
ErrorArchivoVacio(): BE.AppException
ErrorDeFormato(detalle: string): BE.AppException

«abstract»
Servicios.Serializacion.SerializadorAbstracto<T>

+ *Serializar(objeto: T, destino: Stream): void*
+ *Deserializar(origen: Stream): T*
+ Serializar(objeto: T): string
+ Deserializar(texto: string): T
+ GuardarArchivo(objeto: T, ruta: string): void
+ LeerArchivo(ruta: string): T «usa»
- SinContenido(contenido: byte[]): bool

Servicios.Serializacion.SerializadorJson<T>

+ FormatoFecha: string {readOnly}
- ProfundidadMaxima: int {readOnly}
- LargoMaximoTexto: int {readOnly}

«Property»
+ Extension: string
+ Formato: string

- Crear(): DataContractJsonSerializer
+ Serializar(objeto: T, destino: Stream): void
+ Deserializar(origen: Stream): T

Servicios.Serializacion.SerializadorXml<T>

- Serializador: XmlSerializer

«Property»
+ Extension: string
+ Formato: string

+ Serializar(objeto: T, destino: Stream): void
+ Deserializar(origen: Stream): T

«enumeration»
Servicios.ErrorNumero

Ninguno
Vacio
UsarComaDecimal
Formato

«static»
Servicios.FabricaBitacora

- _sistema: Func<IRegistroBitacora>
- _negocio: Func<IRegistroBitacoraNegocio>

«Property»
+ Sistema: Func<IRegistroBitacora>
+ Negocio: Func<IRegistroBitacoraNegocio>

+ CrearSistema(): IRegistroBitacora
+ CrearNegocio(): IRegistroBitacoraNegocio

«static»
Servicios.GeneradorCredenciales

- CarpetaCredenciales: string {readOnly}
- Mayusculas: char[]
- Minusculas: char[]
- Numeros: char[]
- Simbolos: char[]
- Todos: char[]
- CaracteresClave: char[]

+ GenerarContrasena(): string
+ ExportarCredenciales(username: string, contrasena: string): string
+ GenerarClaveRecuperacion(): string
+ ExportarClavesRecuperacion(claves: System.Collections.Generic.IList<string>): string
+ EscribirPrivado(ruta: string, contenido: string): void
- ObtenerCarpeta(): string
- ElegirAleatorio(rng: RandomNumberGenerator, chars: char[]): char
- ObtenerIndice(rng: RandomNumberGenerator, max: int): int

«Create»
- GeneradorCredenciales()

«interface»
Servicios.IRegistroBitacoraNegocio

+ Registrar(tipo: BE.TipoEventoNegocio, descripcion: string, idPedido: int?, idPrenda: int?, idCliente: int?): void

Servicios.BitacoraNegocio

- dal: DAL.BitacoraNegocio
+ LargoDescripcion: int {readOnly}

+ Registrar(tipo: TipoEventoNegocio, descripcion: string, idPedido: int?, idPrenda: int?, idCliente: int?): void
+ ObtenerTodos(): DataTable
+ BuscarPorFiltros(desde: DateTime?, hasta: DateTime?, tipo: string, idCliente: int?, idPedido: int?): DataTable

«interface»
Servicios.IRegistroBitacora

+ Registrar(modulo: string, actividad: string, criticidad: BE.Criticidad): void
+ RegistrarSinSesion(modulo: string, actividad: string, criticidad: BE.Criticidad, idUsuario: int?, detalle: string): void

Servicios.Bitacora

- bitacoraDAL: DAL.Bitacora
- IP_DESCONOCIDA: string {readOnly}
+ LargoModulo: int {readOnly}
+ LargoActividad: int {readOnly}
+ LargoDetalle: int {readOnly}
+ LargoIP: int {readOnly}

~ Ajustar(b: BE.Bitacora): BE.Bitacora
+ Registrar(modulo: string, actividad: string, criticidad: Criticidad): void
+ RegistrarSinSesion(modulo: string, actividad: string, criticidad: Criticidad, idUsuario: int?, detalle: string): void
+ ObtenerTodos(): DataTable
+ ObtenerUltimosNDias(dias: int): DataTable
+ BuscarPorFiltros(desde: DateTime?, hasta: DateTime?, idUsuario: int, actividad: string, criticidad: int): DataTable
+ ObtenerIPLocal(): string

«static»
Servicios.LectorNumeros

- Numero: Regex
- EnteroConMiles: Regex
- PuntoDecimal: Regex

+ TryLeer(texto: string, valor: decimal*, error: ErrorNumero*): bool
+ Leer(texto: string): decimal?

«static»
Servicios.TextoSeguro

- Sufijo: string {readOnly}

+ Recortar(texto: string, largoMaximo: int): string

«usa»
+_observers
0..*
+_idio++Idic
tual
0..1
+_corpus
0..*
+_negocio
1
++Sistema
1